MICRO PROJECT REPORT

FULL STACK DEVELOPMENT

FIXFLOW: COLLEGE EQUIPMENT REPAIR TRACKER

Submitted by:

Srinath K

2511021060261

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

ARTIFICIAL INTELLIGENCE AND FUTURE TECHNOLOGIES

(AIFT)

Under the guidance of

Vaibhav Giri

Technical Mentor, ASAC



Alliance School of Advanced Computing

Alliance University, Bengaluru

October-2026



ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the report on *FIXFLOW: COLLEGE EQUIPMENT REPAIR TRACKER* is submitted by Srinath K [Uni. Roll No. 2511021060261] in partial fulfilment for the award of the degree of Bachelor of Technology (Artificial Intelligence and Future Technologies) of Alliance University, is a bonafide work accomplished under the supervision and guidance of Vaibhav Giri, Technical Mentor, ASAC. This report embodies the study and implementation work carried out for the Full Stack Development microproject during the academic submission period of October 2026.

Vaibhav Giri
Technical Mentor

ALLIANCE SCHOOL OF ADVANCED COMPUTING

DECLARATION

I hereby declare that the report titled *FIXFLOW: COLLEGE EQUIPMENT REPAIR TRACKER*, submitted by me in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence and Future Technologies (AIFT) at Alliance University, is a record of my original academic project work carried out on the *FixFlow College Equipment Repair Tracker*. I further confirm that the technical descriptions, diagrams, screenshots, and conclusions in this report are based on the supplied project implementation and the working application evidence provided for this submission.

Name of the Student	University Registration Number	Signature
Srinath K	2511021060261	*Srinath*

Acknowledgement

I would like to express my heartfelt gratitude to everyone who contributed to the successful completion of this project titled *FIXFLOW: COLLEGE EQUIPMENT REPAIR TRACKER*.

First and foremost, I extend my sincere thanks to my mentor and guide, Vaibhav Giri, Technical Mentor, ASAC, for guidance, encouragement, and academic support throughout the development and documentation of the project.

I am grateful to the faculty members and peers whose technical feedback and suggestions helped in refining the implementation. The FixFlow project provided practical exposure to frontend development, backend API design, relational database management, validation, role-aware workflows, and campus equipment repair tracking.

Finally, I would like to thank everyone who supported me directly or indirectly during the completion of this academic work.

Thank you.

Srinath K

TABLE OF CONTENTS

Contents	Page No.
Acknowledgement	4
Abstract	6
Chapter I – Introduction	7
Problem Statement	7
Objectives	7
Theory	8
Chapter II – System Configuration	9
System Architecture	9
Frontend Implementation	10
Backend and Database Implementation	11
Representative Source Code	12
Chapter III – Results	13
Working Application Screenshots	13
Testing and Subject Mapping	16
Chapter IV – Conclusion	17
References and Appendix	18

Abstract

FixFlow is a full-stack campus equipment repair tracking application designed to organize the reporting, assignment, monitoring, and closure of maintenance complaints in a college environment. It replaces scattered messages and paper-based logs with a centralized system where a fault receives a unique reference and can be followed through a defined repair workflow.

The user interface is implemented with React and Vite. React Router separates the main application pages, while TanStack Query manages server data and mutations. Axios provides a central HTTP client for communicating with the Express API. The interface includes an operations dashboard, fault-reporting form, complaint repository with filters and sorting, complaint investigation page with workflow history, technician roster, equipment inventory, and system diagnostics.

The backend uses Node.js and Express, with Prisma ORM connecting to a PostgreSQL relational database. The data model includes Technician, Equipment, Complaint, and StatusHistory entities, together with enumerated values for complaint status, priority, equipment category, and equipment condition. The project includes request logging, request identifiers, validation and centralized error handling.

This report describes the supplied source implementation and the six provided screenshots. It focuses on the project structure, data model, request flow, repair workflow, user-facing features, and practical application of full-stack development concepts.

Chapter I – Introduction

Problem Statement

College classrooms, laboratories, computer centres, libraries, and network rooms depend on equipment that may fail unexpectedly. When faults are reported through informal messages or paper registers, information can be lost, duplicate requests can occur, responsibility may be unclear, and reporters may not know whether work has started or finished.

FixFlow addresses this problem by maintaining repair tickets in a central repository. Each ticket records the reported problem, equipment details, campus location, urgency, reporter, assigned technician, current status, repair notes, and a history of workflow changes. This allows maintenance work to be tracked from initial report through assignment, repair, resolution, and closure.

Objectives

1. Provide a structured form for reporting hardware faults with equipment, location, priority, and reporter details.
2. Generate a trackable reference for each complaint and show complaints in a searchable repository.
3. Allow authorized administrators to assign technicians and manage repair workflow transitions.
4. Maintain a campus equipment inventory with category, asset label, location, and condition.
5. Present dashboard metrics, status breakdowns, equipment-category counts, and recent activity.
6. Persist records in PostgreSQL through Prisma ORM and expose them through REST API endpoints.
7. Improve user feedback with validation, loading indicators, error messages, confirmation dialogs, and toast notifications.

Scope

The supplied implementation is a campus maintenance tracking application. It demonstrates complaint and asset administration, technician workload visibility, status history, role simulation, and database health diagnostics. The project archive and screenshots are the evidence base for this report; this document does not claim features that are not present in the supplied code.

Theory

React and Component-Based Development
React structures the interface as reusable components and page-level views. In FixFlow, layout components provide a persistent sidebar and top navigation, while pages handle dashboard, reporting, complaints, technicians, inventory, and settings. Small components such as status badges, priority badges, confirmation dialogs, toast messages, loading skeletons, and error boundaries support consistency across the application.

REST API and Client–Server Communication
The browser client communicates with the Express server using HTTP requests. Service modules group API calls by domain, including complaints, technicians, equipment, and dashboard data. A central Axios instance provides a consistent request layer and normalizes API errors before they reach the UI.

TanStack Query and Mutations
TanStack Query is used for asynchronous server state. Query keys organize cached data, while mutations perform create, update, assignment, status-change, and delete operations. Successful mutations invalidate related query keys so the interface can refresh affected lists and detail views. The complaint hook also supports optimistic status updates where implemented.

Relational Database and ORM
PostgreSQL stores the application records. Prisma defines typed models, relations, indexes, and enumerated fields in schema.prisma. The ORM separates application logic from handwritten SQL for most CRUD operations and supports relations between complaints, technicians, equipment, and status-history entries.

Validation, Error Handling and Auditability
Client-side form checks help users correct input before submission; server-side request validation is an additional enforcement layer. Middleware attaches request identifiers, logs request information, and handles errors centrally. StatusHistory stores workflow transitions, including previous and new status, notes, actor, and timestamp, providing an audit trail for repairs.

Chapter II – System Configuration

System Architecture

FixFlow follows a layered client–server design. The React application presents the user interface and obtains data through domain-specific service modules. Those services use Axios to call the Express API. Express routes dispatch requests to controllers and services, which use Prisma to query or update PostgreSQL. The response returns to the client, where query state is refreshed and the relevant components render updated information.

React Pages / Components

↓

Hooks and TanStack Query → Domain Service Modules

↓

Axios HTTP Client → Express Routes / Controllers

↓

Domain Services → Prisma ORM → PostgreSQL

Technology Stack

Layer	Technology	Purpose
Frontend	React, Vite, React Router	Component UI, development build, client-side navigation
Client state / requests	TanStack Query, Axios	Fetch/cache server data, submit mutations, handle HTTP errors
Forms and visuals	React Hook Form, Lucide, Recharts	Form state, icons, dashboard charts
Backend	Node.js, Express	REST endpoints, domain logic, middleware
Database access	Prisma ORM	Schema, relations, database operations and migrations
Database	PostgreSQL	Persistent storage for tickets, assets, technicians and audit history

Project Structure

- client/src/app — application setup, router and query client.
- client/src/pages — dashboard, report form, complaint list/detail, technicians, equipment and settings.
- client/src/hooks and client/src/services — query/mutation hooks, query keys, and API service functions.
- server/src/modules — complaint, technician, equipment and dashboard modules with routes, controllers and services.
- server/prisma — Prisma schema, migration and seed data.
- server/src/middleware — request ID, request logging, request validation and centralized error handling.

Frontend Implementation

Application Shell and Navigation

The main layout keeps navigation and the top bar consistent across pages. The sidebar links to Dashboard, Report a Fault, All Complaints, Technicians, Equipment Inventory, and Settings & System. The top bar shows the simulated role and PostgreSQL connection indicator. React Router defines the application routes and uses lazy-loaded page modules to reduce the initial bundle.

Operations Dashboard

DashboardPage presents high-level maintenance metrics such as total faults, pending tickets, in-progress work, resolved or closed tickets, and high/critical issues. It includes a status distribution chart, equipment-category chart, recent fault complaints, and a recent activity feed. Dashboard service functions obtain aggregated information from the backend rather than requiring the page to calculate all totals from static data.

Fault Reporting Form

ReportComplaintPage gathers a complaint title, equipment category and model, optional asset label, detailed problem description, building or block, room or laboratory, priority, reporter name, and college email. Required fields are marked in the form. The page offers a quick demo-fill option and reset/submit controls. This structure gives maintenance staff enough context to identify the affected equipment and assess urgency.

Complaint Repository and Investigation

ComplaintsListPage provides text search, status, priority, and category filters, sorting, clear controls, and paginated results. Each row shows the ticket reference, equipment and fault summary, category, location, priority, status, technician, and a View action. ComplaintDetailPage displays the fault description, equipment/location facts, assigned technician, reporter details, and workflow audit history. It supports technician assignment, permitted status changes, and administrator deletion with confirmation.

Technician and Asset Pages

TechniciansPage shows staff specializations, availability, active repairs and completed work, with add/edit/delete actions for administrators. EquipmentPage organizes campus assets by category and shows the asset label, location, condition and number of associated complaints. SettingsPage provides a real API/database health check and role selection for the demonstration environment.

Backend and Database Implementation

Express Application and Middleware

The backend is an Express application. The supplied source includes request-ID middleware, request logging, request validation, and a central error handler. These middleware functions help correlate requests, record response information, standardize validation errors, and convert known database errors into useful API responses. The server entry point starts the application and reports startup diagnostics.

Domain Modules and API Responsibilities

- Complaints module: creates and retrieves fault tickets, filters/listing data, updates workflow status, assigns technicians, stores repair notes, and manages complaint history.
- Technicians module: lists staff, creates and updates technician profiles, tracks specializations and availability, and supports deletion rules related to active repairs.
- Equipment module: registers and updates assets, filters by category, and stores labels, building/room, and condition.
- Dashboard module: aggregates complaint totals and status/category summaries and supplies recent activity and health information.

Relational Data Model

The Prisma schema defines four main models. Technician records contain name, unique email, specialization and availability. Equipment records contain name, category, unique asset label, building, room and condition. Complaint records contain the unique reference, title, description, equipment and location details, reporter data, priority, status, repair notes, and optional technician/equipment relations. StatusHistory stores each workflow event with old status, new status, note, actor and timestamp.

Key Relationships and Constraints

- A technician can be related to multiple complaints; the technician relation is optional and is set to null if the technician is deleted.
- An equipment asset can be associated with multiple complaints; the optional equipment relation is set to null if an asset is deleted.
- A complaint can have multiple status-history records; history rows are removed when the parent complaint is deleted.
- Unique constraints protect technician email, equipment asset label, and complaint reference.
- Indexes are defined for fields frequently used in filtering or ordering, including status, priority, category, technician, creation date, and history timestamps.

Workflow States

The complaint status enum contains OPEN, ASSIGNED, IN_PROGRESS, ON_HOLD, RESOLVED, and CLOSED. The detail page explicitly maps allowed transitions: open tickets can be assigned, started, or put on hold; assigned tickets can be started, held, or reopened to open; in-progress tickets can be held or resolved; on-hold tickets can resume, resolve, or return to assigned; resolved tickets can be closed or reopened to in-progress; closed tickets can only be reopened to open.

Representative Source Code

1. Complaint Workflow State Transitions

```
const ALLOWED_STATUS_TRANSITIONS = {
  OPEN: ["ASSIGNED", "IN_PROGRESS", "ON_HOLD"],
  ASSIGNED: ["IN_PROGRESS", "ON_HOLD", "OPEN"],
  IN_PROGRESS: ["ON_HOLD", "RESOLVED"],
  ON_HOLD: ["IN_PROGRESS", "RESOLVED", "ASSIGNED"],
  RESOLVED: ["CLOSED", "IN_PROGRESS"],
  CLOSED: ["OPEN"],
};
```

This transition map prevents the interface from offering arbitrary status changes. It makes the intended repair lifecycle explicit and easier to review.

2. Client API Error Normalization

```
// Central Axios response handling (simplified from client/src/services/api.js)
api.interceptors.response.use(
  (response) => response,
  (error) => {
    const payload = error.response?.data;
    return Promise.reject(
      payload?.error || {
        code: "NETWORK_ERROR",
        message: error.message || "Unable to reach the API.",
      }
    );
  }
);
```

3. Data Model Excerpt

```
model StatusHistory {
  id          String          @id @default(uuid())
  complaintId String
  complaint   Complaint       @relation(fields: [complaintId], references: [id], onDelete:
Cascade)
  oldStatus   ComplaintStatus?
  newStatus   ComplaintStatus
  note        String?
  changedBy   String?         @default("System")
  createdAt   DateTime        @default(now())
}
```

These representative excerpts are drawn from the supplied implementation, with the API error handling presented in shortened form for readability. The full source code remains in the supplied project archive.

Chapter III – Results

Working Application Screenshots

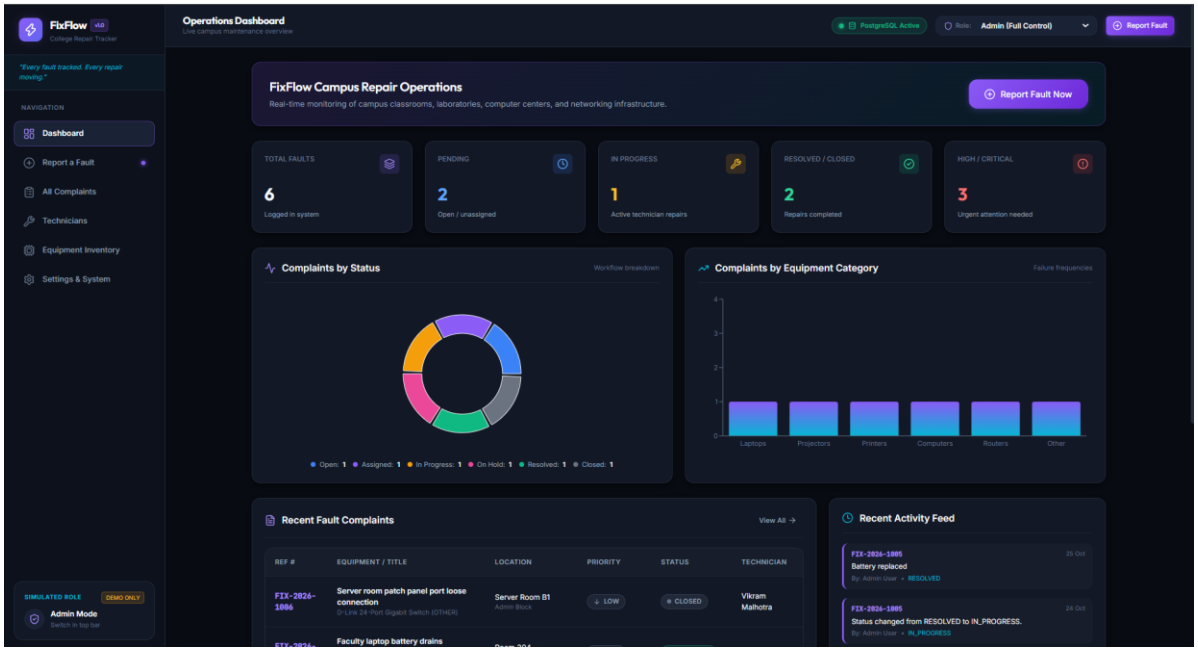


Figure 1. Operations dashboard showing maintenance metrics, complaint status distribution, equipment-category counts, recent complaints, and activity feed.

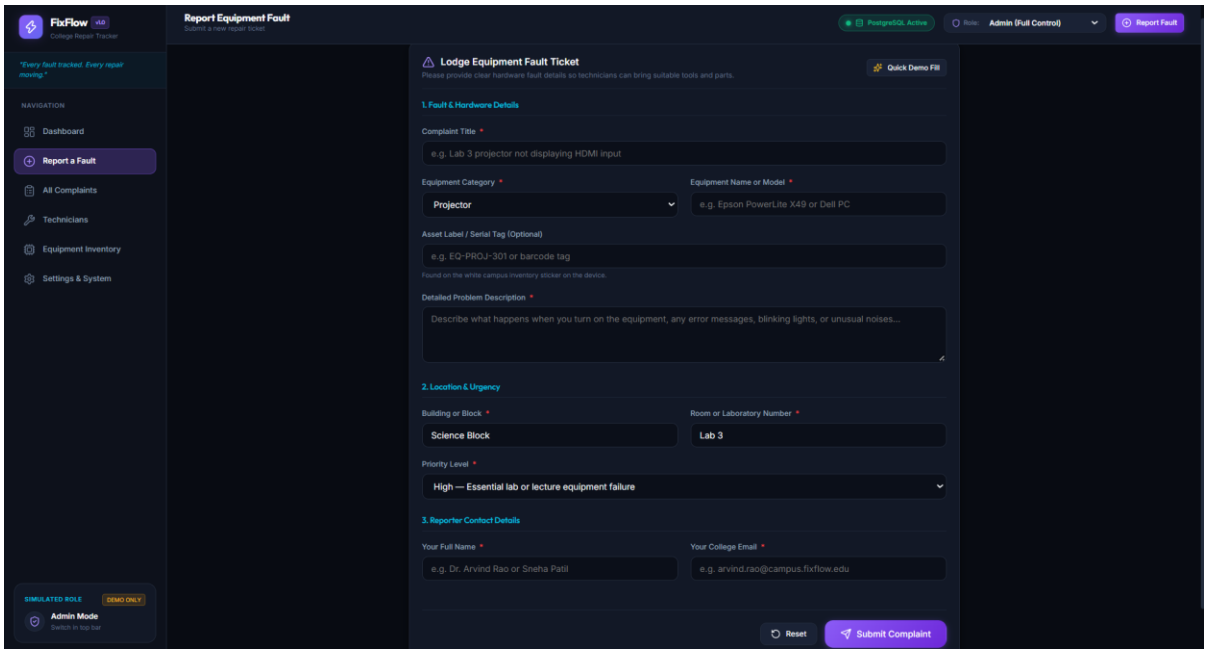


Figure 2. Report Equipment Fault form showing the required fault details, equipment category, location and urgency, and reporter contact fields.

Working Application Screenshots

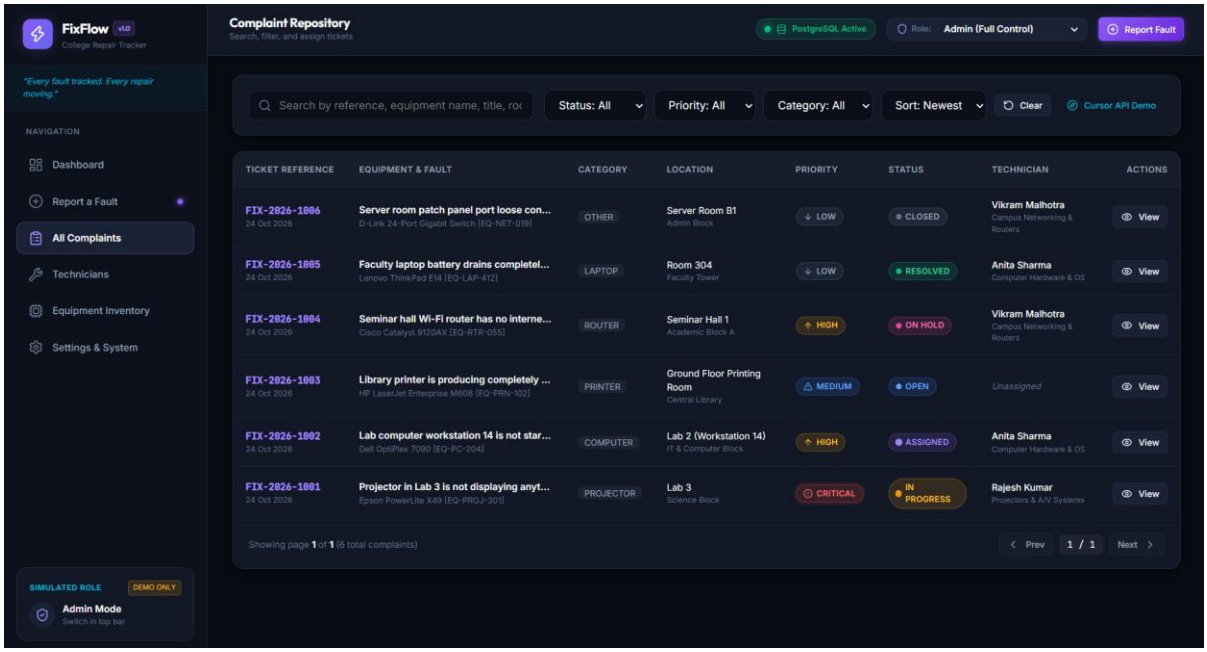


Figure 3. Complaint repository showing search and filter controls, ticket references, equipment, category, location, priority, status, technician and actions.

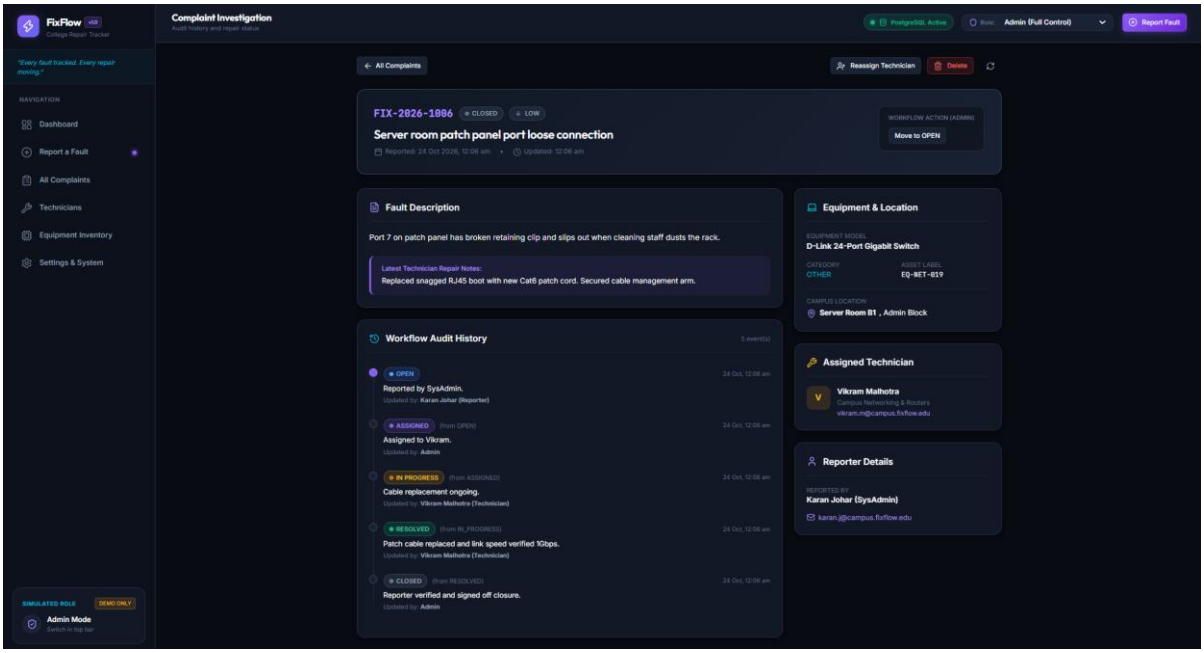


Figure 4. Complaint investigation page showing the selected ticket, repair description, equipment/location, assigned technician, reporter details and workflow audit history.

Working Application Screenshots

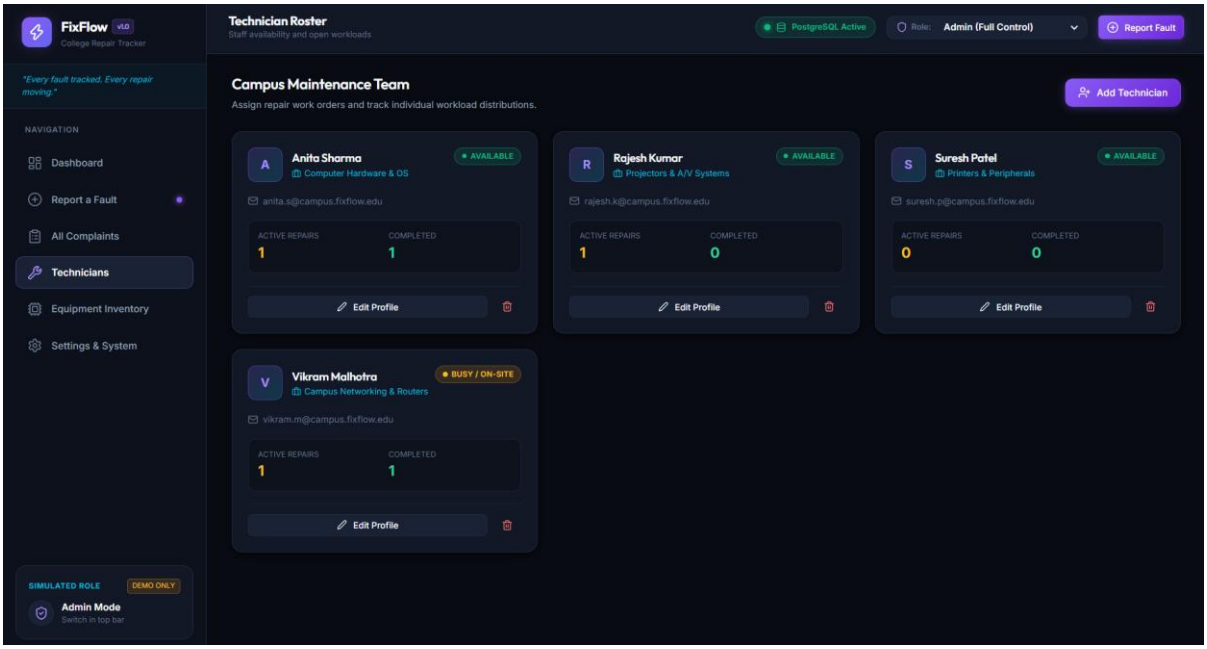


Figure 5. Technician roster showing staff specialization, availability, active repairs, completed repairs and profile management controls.

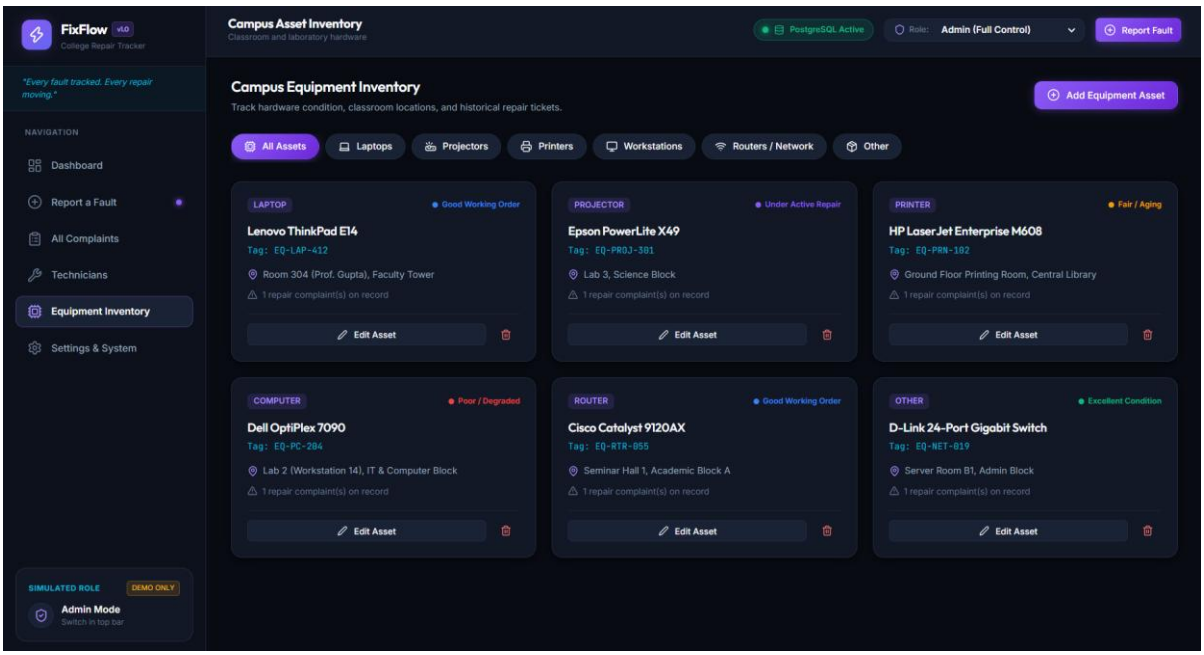


Figure 6. Campus asset inventory showing equipment categories, asset labels, locations, condition indicators and asset management actions.

Testing and Subject Mapping

The following mapping connects Full Stack Development concepts to the actual FixFlow files and the visible application evidence.

Subject Topic	Actual Project File / Component	Implementation Details	Evidence
React components	client/src/pages and client/src/components	Page composition, shared navigation, status/priority badges and feedback components.	Figures 1–6
Routing	client/src/app/router.jsx	Page routes and lazy-loaded page modules.	Source
TanStack Query	client/src/hooks/useComplaints.js; client/src/services/queryKeys.js	Server queries, mutation state, cache invalidation and optimistic status updates.	Source
Axios / REST	client/src/services/api.js; client/src/services/*Service.js	Central HTTP client and domain-specific API calls.	Source
Express backend	server/src/app.js; server/src/modules/*	Routes, controllers, domain services and middleware.	Source
Relational modelling	server/prisma/schema.prisma	Technician, Equipment, Complaint and StatusHistory models with enums and relations.	Source
Workflow / audit trail	ComplaintDetailPage.jsx; StatusHistory model	Permitted status transitions and recorded status-change history.	Figures 3–4 + source
Form handling	ReportComplaintPage.jsx	Fault details, location, urgency and reporter fields.	Figure 2
Inventory / CRUD	EquipmentPage.jsx; equipment service/module	Asset list, categories, condition, add/edit/delete controls.	Figure 6
Technician management	TechniciansPage.jsx; technician service/module	Specialization, availability and workload indicators.	Figure 5
System diagnostics	SettingsPage.jsx; dashboard health endpoint	API and database connectivity check.	Source

Chapter IV – Conclusion

The FixFlow College Equipment Repair Tracker demonstrates how a full-stack application can organize campus maintenance operations through a connected frontend, backend API, and relational database. It brings fault reporting, complaint search, technician assignment, repair workflow tracking, and equipment inventory into one interface.

The React frontend separates pages, reusable interface components, hooks, and service functions. TanStack Query coordinates server-state fetching and mutation results, while Axios provides a consistent HTTP layer. The Express backend groups responsibilities into complaint, technician, equipment, and dashboard modules. Prisma and PostgreSQL provide a structured persistence layer for complaint records, equipment assets, technician profiles, and workflow history.

The complaint status model and audit-history records are important parts of the project. They allow the system to represent repair progress and retain notes about workflow changes. The dashboard and repository help users view overall maintenance activity, while detailed complaint and inventory pages support day-to-day administration.

The supplied screenshots provide visual evidence of the dashboard, reporting form, complaint repository, investigation details, technician roster, and equipment inventory. Together with the source archive, they demonstrate the main user-facing and technical elements of the implementation.

Future Enhancements

- Replace demonstration role simulation with production authentication and server-enforced role-based authorization.
- Add automated unit, integration, and end-to-end tests for the main complaint and inventory workflows.
- Introduce notifications for assignment, status changes, and overdue repair tickets.
- Add reporting/export features for maintenance history, asset lifecycle and technician workload.
- Deploy the application with environment-specific secrets, monitoring, backups and documented operational procedures.

References and Appendix

References

1. Supplied FixFlow – College Equipment Repair Tracker source code archive (client, server, Prisma schema, migrations, and README).
2. React documentation: component-based UI development and rendering concepts.
3. TanStack Query documentation: queries, mutations, cache invalidation, and server-state management.
4. Axios documentation: HTTP client configuration and interceptors.
5. Express documentation: routing, middleware, and REST API construction.
6. Prisma documentation: schema modelling, relations, migrations, and database client.
7. PostgreSQL documentation: relational database concepts and SQL data storage.

Appendix A – Principal Frontend Files

- client/src/app/App.jsx; client/src/app/router.jsx; client/src/app/queryClient.js
- client/src/pages/DashboardPage.jsx; client/src/pages/ReportComplaintPage.jsx
- client/src/pages/ComplaintsListPage.jsx; client/src/pages/ComplaintDetailPage.jsx
- client/src/pages/TechniciansPage.jsx; client/src/pages/EquipmentPage.jsx; client/src/pages/SettingsPage.jsx
- client/src/hooks/useComplaints.js; client/src/services/api.js; client/src/services/queryKeys.js
- client/src/services/complaintService.js; client/src/services/technicianService.js; client/src/services/equipmentService.js; client/src/services/dashboardService.js

Appendix B – Principal Backend Files

- server/src/app.js; server/src/server.js; server/src/config/prisma.js
- server/src/middleware/requestId.js; requestLogger.js; validateRequest.js; errorHandler.js
- server/src/modules/complaints/complaint.routes.js; complaint.controller.js; complaint.service.js
- server/src/modules/technicians/technician.routes.js; technician.controller.js; technician.service.js
- server/src/modules/equipment/equipment.routes.js; equipment.controller.js; equipment.service.js
- server/src/modules/dashboard/dashboard.routes.js; dashboard.controller.js; dashboard.service.js
- server/prisma/schema.prisma; server/prisma/seed.js; server/prisma/migrations/20261024000000_init/migration.sql

The appendix identifies the principal files examined for this report. The complete implementation is contained in the supplied Full Stack project archive.